

Smart E-Commerce Platform

High-Level System Architecture Overview

React.js + Tailwind CSS · Node.js + Express.js · PostgreSQL

Roles: Admin & User | State Management: Context API / local state (no Redux)

Scope: main system components · frontend/backend/database interaction · authentication flow · key modules (Product, Cart, Order, Invoice, Tracking)

1. Main System Components

The platform follows a classic 3-tier architecture: a React SPA (presentation tier), an Express REST API (application/business tier), and PostgreSQL (data tier). State is managed with React's built-in Context API and component state — no Redux — keeping the frontend lightweight for this scope.

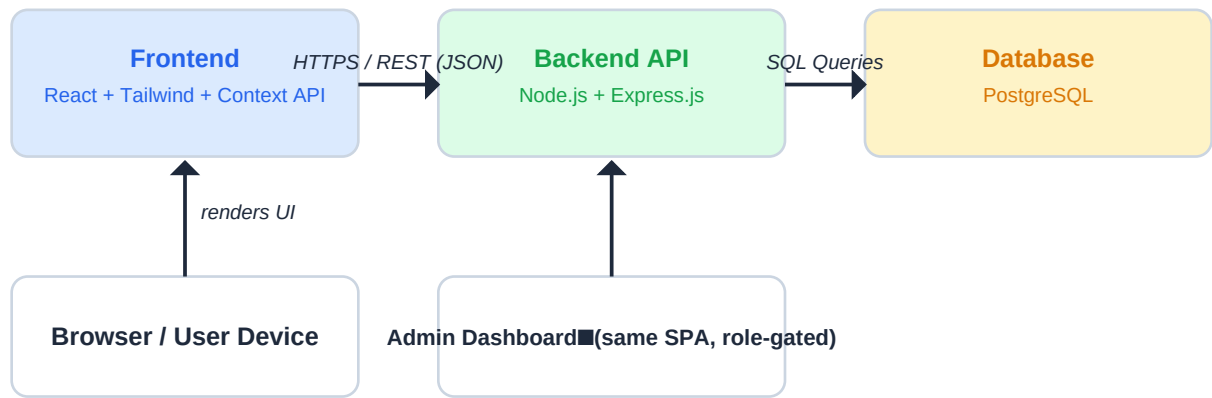


Figure 1 — Three-tier system component overview

1.1 Component Responsibilities

Component	Responsibility
Frontend (React + Tailwind)	Renders product catalog, cart, checkout, order tracking, and admin screens; calls backend via REST; manages UI state locally/Context API.
Backend API (Express)	Exposes REST endpoints; enforces authentication & role-based access; contains business logic for products, cart, orders, invoices, tracking.
Database (PostgreSQL)	Persists users, products, carts, orders, order status history, and invoice records; enforces relational integrity.
Admin Dashboard	Same React app, gated by role; provides product management and order-status control screens.

2. Frontend ■ Backend ■ Database Interaction

Every user action in the browser triggers an API call from the frontend to the backend, which validates the request, applies business rules, and reads/writes PostgreSQL — then returns a JSON response that updates the UI.

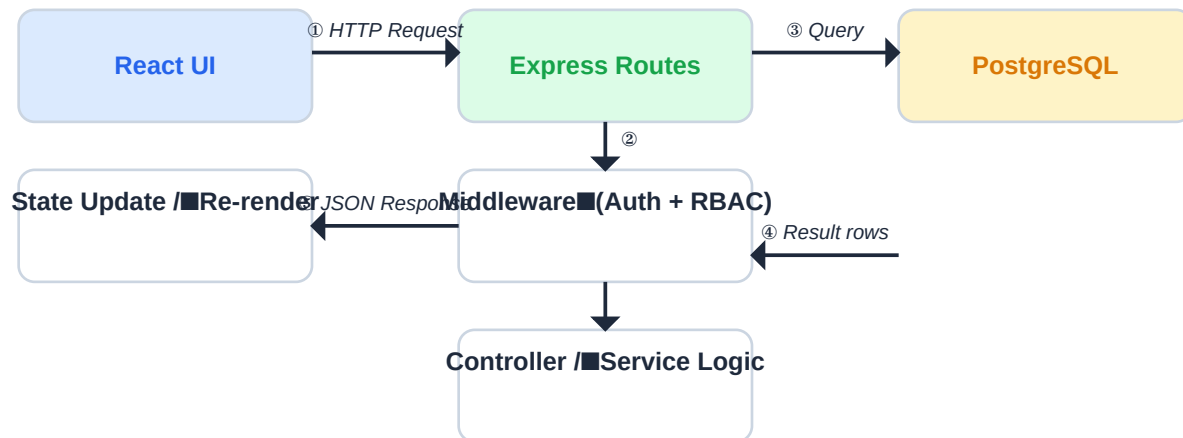


Figure 2 — Request/response lifecycle across the stack

2.1 Interaction Summary

- ① Frontend sends an HTTPS request (with JWT in the Authorization header) to an Express route.
- ② The request passes through authentication and role-based access middleware before reaching business logic.
- ③–④ The service layer queries PostgreSQL and receives structured result rows.
- ⑤ The API returns a JSON response; React updates local/Context state and re-renders the affected view.

3. Authentication & Authorization Flow

Authentication uses JWT-based sessions. Passwords are hashed before storage, and every protected request carries a bearer token that identifies the user and role, letting the same middleware layer separate Admin-only actions from User actions.

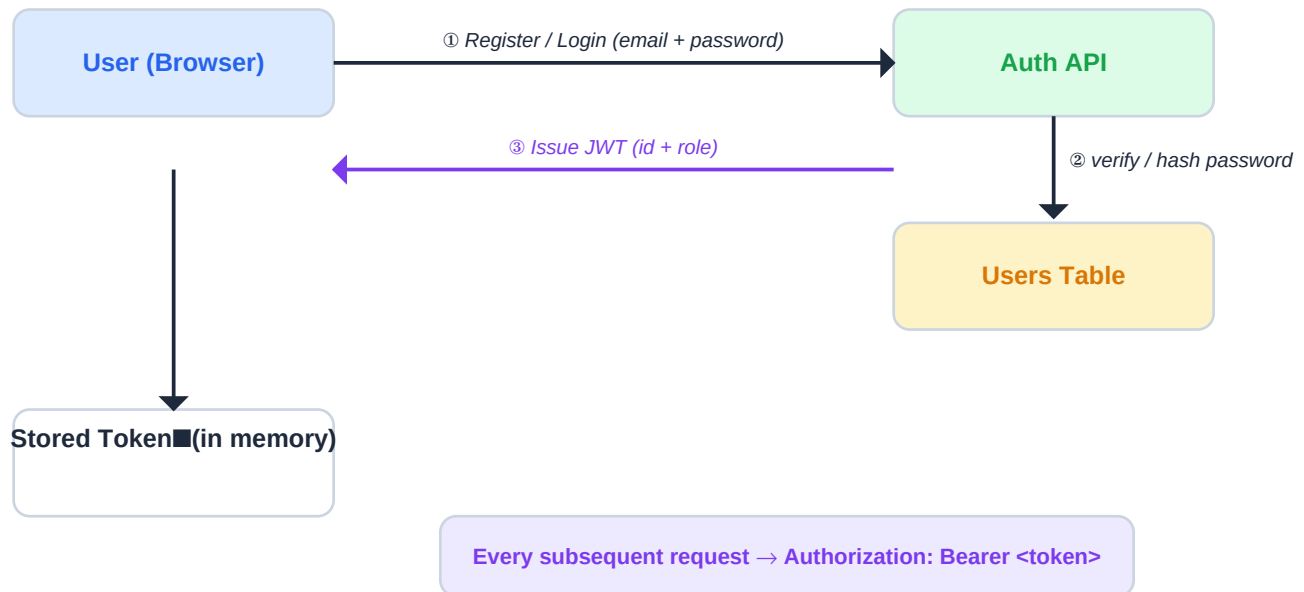


Figure 3 — Authentication sequence

3.1 Role-Based Access

- **Registration/Login** — user submits credentials; backend hashes/verifies the password and issues a signed JWT containing user ID and role (Admin/User).
- **Token storage** — frontend keeps the token in memory/local storage and attaches it to every API call.
- **Authorization middleware** — backend verifies the token on each request and checks the role claim before allowing access to Admin-only routes (e.g. product management, order-status updates).
- **Session expiry** — expired/invalid tokens are rejected, prompting re-login.

4. Key Modules

The backend is organized into cohesive modules, each owning one business capability. Modules communicate through the shared database and, where needed, call one another's internal services directly (e.g., Order module updates Product stock).

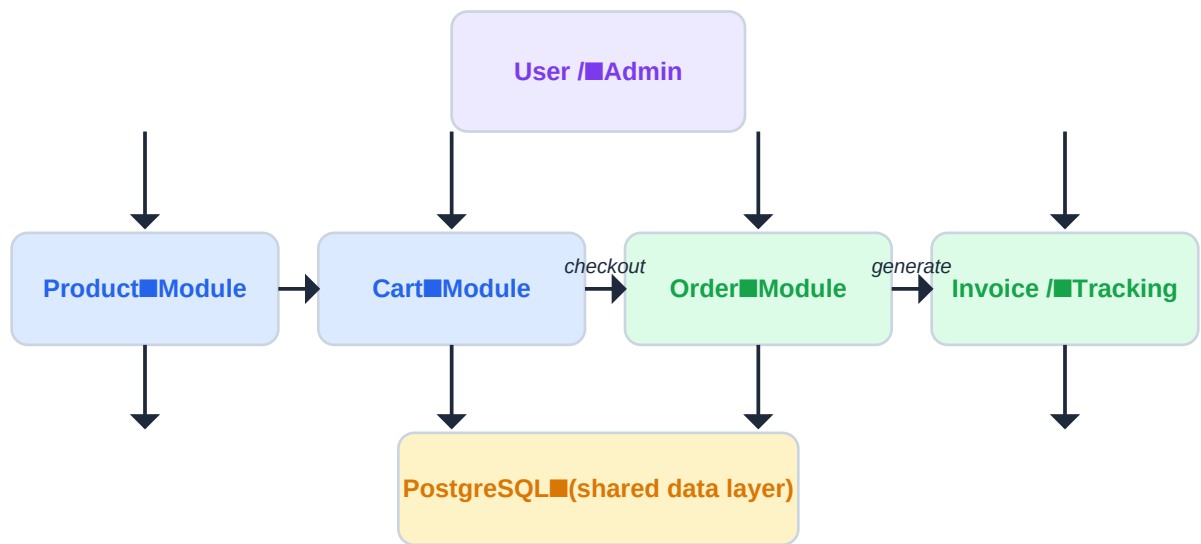


Figure 4 — Module map and interaction

Module	Responsibility
Product Module	Admin adds/edits/deletes/updates products (name, price, stock, category, image). Users browse and search the catalog.
Cart Module	Lets a logged-in User add/update/remove products before checkout; cart contents feed directly into order creation.
Order Module	Handles checkout/order placement, stores order details, and manages the order status lifecycle (Pending → Shipped → Delivered, etc.) as updated by Admin.
Invoice Module	Automatically generates an invoice once an order is placed; User can view/download it.
Tracking Module	Surfaces the current and historical order status to the User in real time, reflecting Admin status updates.

5. Order Lifecycle & Status Tracking (Overview)

Once a User checks out, the order moves through a defined sequence of statuses. Admin advances the status; the User's tracking view reflects each change.

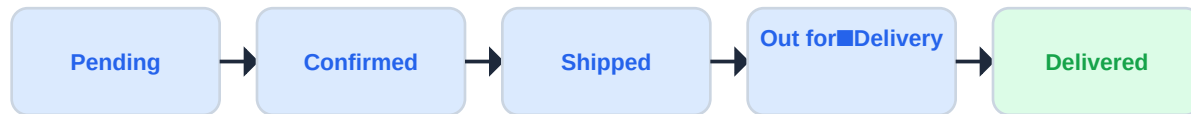


Figure 5 — Order status progression (Admin-driven, User-visible)

Note: An order may also branch to **Cancelled** from an early stage. Each status change is timestamped so the User's tracking screen shows a running history.

End of Document — High-Level Architecture Overview v1.0